

Ejercicios Prácticos: Sistema Bancario "BootBank"

Objetivo: Modelar las entidades clave de un sistema bancario básico, aplicando DDL con integridad y relaciones del mundo real.

Nivel 1: Entidades Fundamentales y Restricciones Básicas

Ejercicio 1: "La Base de Todo: Los Clientes"

Crea la tabla `clientes`. Es nuestro punto de partida.

- `cliente_id` (entero, será nuestra clave primaria)
- `rut` (texto, 10 caracteres, debe ser único e irrepetible en el sistema)
- `nombre` (texto, no nulo)
- `apellido` (texto, no nulo)
- `fecha_nacimiento` (fecha)
- `email` (texto, debe ser único y no nulo)

Asegúrate de definir la PRIMARY KEY correctamente.

Ejercicio 2: "El Producto Principal: Las Cuentas"

Crea la tabla `cuentas`. Cada cliente puede tener varias.

- `cuenta_id` (entero, PRIMARY KEY)
- `cliente_id` (entero, esta columna vinculará con la tabla `clientes`)
- `numero_cuenta` (texto, 10 caracteres, debe ser único en todo el banco)
- `tipo_cuenta` (texto, solo puede ser 'AHORRO', 'CORRIENTE' o 'VISTA')
- `saldo` (numérico con decimales, no nulo. El saldo inicial DEBE ser 0.0 o positivo)
- `fecha_apertura` (fecha, por defecto, la fecha del sistema)

Aquí solo crea la tabla. La relación (FOREIGN KEY) la haremos después.

Nivel 2: Estableciendo Relaciones Críticas (FOREIGN KEYS)

Ejercicio 3: "Vinculando Clientes con sus Cuentas"

Ahora, establece la relación entre `cuentas` y `clientes`.

1. Agrega una **FOREIGN KEY** en la columna `cliente_id` de la tabla `cuentas` que haga referencia a `cliente_id` en `clientes`.

2. Define que si un cliente es eliminado de la base de datos (por error administrativo, por ejemplo), NO se podrán borrar sus cuentas si existen registros. Usa `ON DELETE RESTRICT`. Esto protege la integridad histórica.

Ejercicio 4: "Registro de Movimientos: El Libro Mayor"

Crea la tabla `movimientos`. **Esta es la tabla más importante, es inmutable.** Registra cada depósito, retiro o transferencia.

- `movimiento_id` (entero, PRIMARY KEY)
- `cuenta_id` (entero, no nulo)
- `tipo` (texto, solo puede ser 'DEPOSITO', 'RETIRO', 'TRANSFERENCIA_ENVIADA', 'TRANSFERENCIA_RECIBIDA')
- `monto` (numérico con decimales, siempre positivo, no nulo)
- `fecha_hora` (marca de tiempo, por defecto, el momento exacto de la inserción)
- `cuenta_destino` (texto, nullable, para guardar el número de cuenta externa en transferencias)

Piensa: ¿Cómo relacionarías esta tabla con `cuentas`? Lo haremos en el siguiente paso.

Nivel 3: Integridad Referencial y Comportamiento (ON DELETE/UPDATE)

Ejercicio 5: "La Huella Digital de cada Transacción"

Establece la relación entre `movimientos` y `cuentas`.

1. Agrega la **FOREIGN KEY** desde `movimientos.cuenta_id` hacia `cuentas.cuenta_id`.
2. En este caso, ¿qué debe pasar si alguien intenta borrar una cuenta que ya tiene movimientos históricos? **Los movimientos NUNCA se deben borrar.** Configura la clave foránea con `ON DELETE RESTRICT`.
3. Agrega también una restricción `CHECK` para que el `monto` sea siempre mayor a 0.

Ejercicio 6: "Modelando los Créditos"

Crea la tabla `creditos`. Un cliente puede tener varios créditos vigentes.

- `credito_id` (entero, PRIMARY KEY)
- `cliente_id` (entero, FOREIGN KEY a `clientes`)
- `monto_total` (numérico, > 0)
- `monto_pendiente` (numérico, >= 0)
- `tasa_interes` (numérico, > 0)
- `estado` (texto, solo puede ser 'APROBADO', 'DESEMBOLSADO', 'PAGADO', 'VENCIDO')
- `fecha_desembolso` (fecha, puede ser nula si el crédito solo está 'APROBADO')

Aquí hay una lógica de negocio: `monto_pendiente` nunca puede ser mayor que `monto_total`. Lo resolveremos con un `CHECK`.

Nivel 4: Alteraciones y Mantenimiento del Sistema (ALTER TABLE)

Ejercicio 7: "Nuevo Requisito de Negocio: Límite de Crédito"

El área de Riesgos pide agregar un nuevo atributo a los clientes.

1. Agrega una columna `limite_credito_aprobado` (numérico) a la tabla `clientes`. Por defecto, debe ser 1000000 para clientes nuevos.
2. Modifica la tabla `creditos` para agregar una columna `fecha_aprobacion` (fecha).
3. **Desafío:** Agrega una restricción `CHECK` en la tabla `creditos` para garantizar que `monto_pendiente <= monto_total`.

Ejercicio 8: "Corrección de Diseño: Tipos de Cuenta Más Específicos"

El negocio se expande. Necesitamos más detalle en cuentas.

1. Cambia la columna `tipo_cuenta` para que, en lugar de texto libre, sea un `ENUM` (o un `CHECK` más estricto) que permita: 'AHORRO_NORMAL', 'AHORRO_VIP', 'CORRIENTE_PERSONAL', 'CORRIENTE_EMPRESARIAL', 'PLAZO_FIJO_30D', 'PLAZO_FIJO_90D'.
2. Renombra la columna `saldo` a `saldo_disponible` para mayor claridad.

Nivel 5: Desafío de Diseño y Eliminación Segura

Ejercicio 9: "Tablas de Catalogos (Lookup Tables)"

Para profesionalizar el modelo, crea tablas auxiliares que eviten datos mal escritos.

1. Crea una tabla `tipos_movimiento` con: `tipo_id` (PK), `nombre` (UNIQUE, ej: 'DEPOSITO'), `descripcion`.
2. **Borra la columna `tipo` de la tabla `movimientos`** (usa `ALTER TABLE ... DROP COLUMN`).
3. Agrega una nueva columna `tipo_id` a `movimientos` y establécela como `FOREIGN KEY` hacia `tipos_movimiento.tipo_id`. Esto normaliza el diseño.

Ejercicio 10: "Procedimiento de Baja de Cliente (Borrado Seguro)"

Simula un escenario de mantenimiento.

1. Intenta escribir un comando `DELETE` para borrar un cliente que tenga cuentas y movimientos asociados. ¿Qué sucede? (Debe fallar por la regla `RESTRICT`).
2. Cambia la `FOREIGN KEY` entre `cuentas` y `clientes` para que use `ON DELETE SET NULL`.
3. Conclusión práctica: En sistemas críticos, no se usa `DELETE`. Se agrega una columna `activo` `BOOLEAN` `DEFAULT true` y se hacen "bajas lógicas" con `UPDATE`.